CS & IT ENGINEERING



Algorithms

Sorting Algorithms



Lecture No.- 02

By- Aditya sir

Recap of Previous Lecture



- Topic
- Topic

Imp Terminologies

→ Bubble Sort

↳ Algol

Topics to be Covered



Topic

Topic

Topic

Bubble Sort

Selection Sort

Insertion Sort



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir:

https://t.me/AdityaSir_PW



Topic : Sorting Algorithms

Algo 1: 1-based indexing

```
Algo BubbleSort(A, n)
{
    for (pass = 1; pass <= (n–1); pass++)
    {
        for (j = 1; j <= (n–1); j++)
        {
            if (A[j] > A[j+1])
            {
                swap(A[j], A[j+1])
            }
        }
    }
}
```



Topic : Sorting Algorithms

Bubble Sort:

Algo1 :

TC analysis

	No. of Comp	No. of Swaps
B.C	$(n-1)^2$	0
W.C	$(n-1)^2$	$\frac{n(n-1)}{2}$



Topic : Sorting Algorithms

Algo 2: Better version of Bubble sort (Algo 1)

```
Algo BubbleSort(A, n)
{
    for (pass = 1; pass <= (n–1); pass++)
    {
        for (j = 1; j <= (n–pass); j++)
        {
            if (A[j] > A[j+1])
            {
                swap(A[j], A[j+1])
            }
        }
    }
}
```

$j <= (n-1)$

$j \leq n - pass$

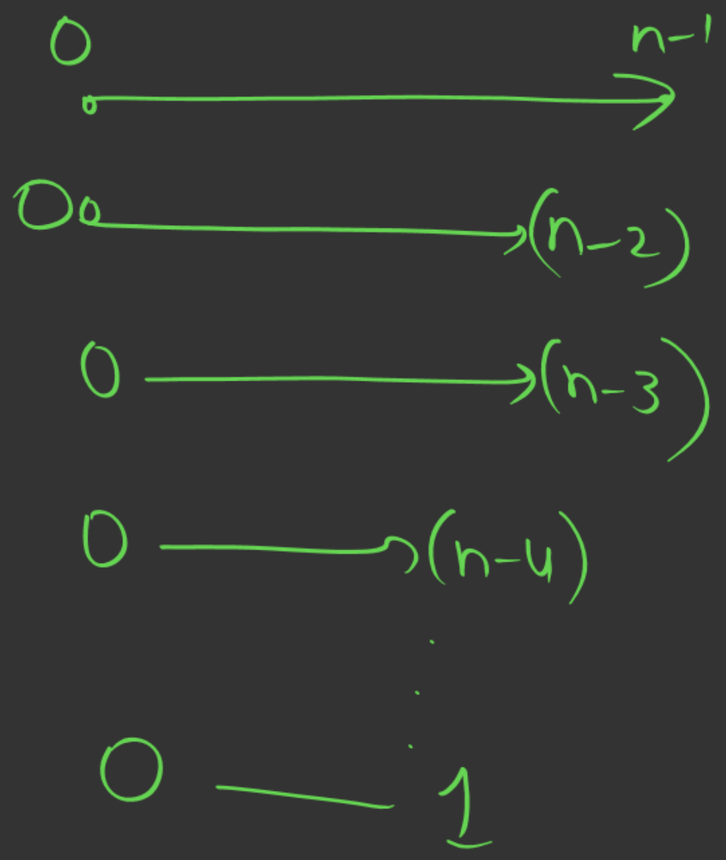


Topic : Sorting Algorithms

Time Complexity analysis of Algo1 & Algo 2:

Algo 1:

	No. of Comparisons	No. of Swaps (inversions)
Best case Ascending order	$(n-1) * (n-1)$ $=(n-1)^2$	0
Worst Case Descending order	$(n-1) * (n-1)$ $=(n-1)^2$	$\frac{n(n-1)}{2}$



$$1+2+3\cdots(n-2)+(n-1)$$

$$= \frac{n(n-1)}{2}$$



Topic : Sorting Algorithms

Time Complexity analysis of Algo1 & Algo 2:

Algo 2:

	No. of Comparisons		No. of Swaps (inversions)	
Best case Ascending order	$\frac{n(n-1)}{2}$	+	0	$\Omega(n^2)$
Worst Case Descending order	$\frac{n(n-1)}{2}$	+	$\frac{n(n-1)}{2}$	$O(n^2)$

$\Rightarrow \Theta(n^2)$



Topic : Sorting Algorithms

We need to reduce the unnecessary comparisons in Best Case.

[2 5 7 10 15 17]

Best Case → i/p is already sorted → Swaps = 0



Topic : Sorting Algorithms

Issue with Algo1 and Algo2:

A:	10	20	25	40

pass 1 :- [10 | 20 | 25 | 40] →

0 Swaps → already Sorted.

Algo2 : n = 4

(n-1) passes

3 passes



Topic : Sorting Algorithms

Optimized Bubble Sort:

- If there are no swaps in any of the pass, then the array has got sorted already.

Idea



Topic : Sorting Algorithms

Algo 3: Optimized Bubble sort

Algo 3

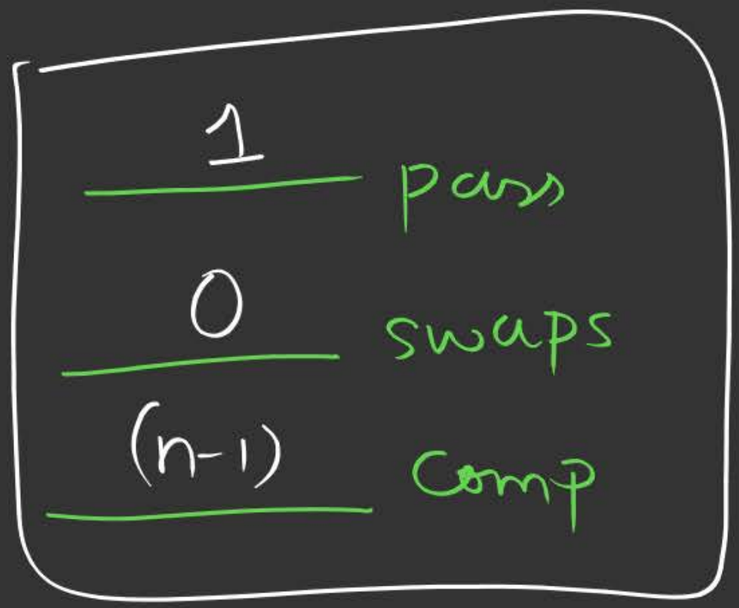
```
Algo BubbleSort(A, n)
{
    for (pass = 1; pass <= (n–1); pass++)
    {
        Flag = 0
        for (j = 1; j <= (n–pass); j++)
        {
            if (A[j] > A[j+1])
            {
                swap(A[j], A[j+1])
                flag = 1            // inversion was present
            }
        }
        if (flag == 0) { break; } // means there are not any inversions
    }
}
```

Pass 1

Pass 2

Pass 3

Algo3 B.C





Topic : Sorting Algorithms

Time Complexity analysis of Algo3 Bubble Sort:

Best Case: 1 pass, 0 swaps, (n–1) comparisons

	No. of Comparisons	No. of Swaps (inversions)
Best case Ascending order	$(n-1)$	0
Worst Case Descending order	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$

$\rightarrow \Omega(n)$

$\rightarrow O(n^2)$

$\neq \Theta(n^2)$



Topic : Sorting Algorithms

~~Space~~ Time Complexity analysis of Algo3 Bubble Sort:

Best Case: $T(n) = (n-1) + C \rightarrow O(n)$ BC

Worst Case: $T(n) = T(n-1) + (n-1) \rightarrow O(n^2)$ WC

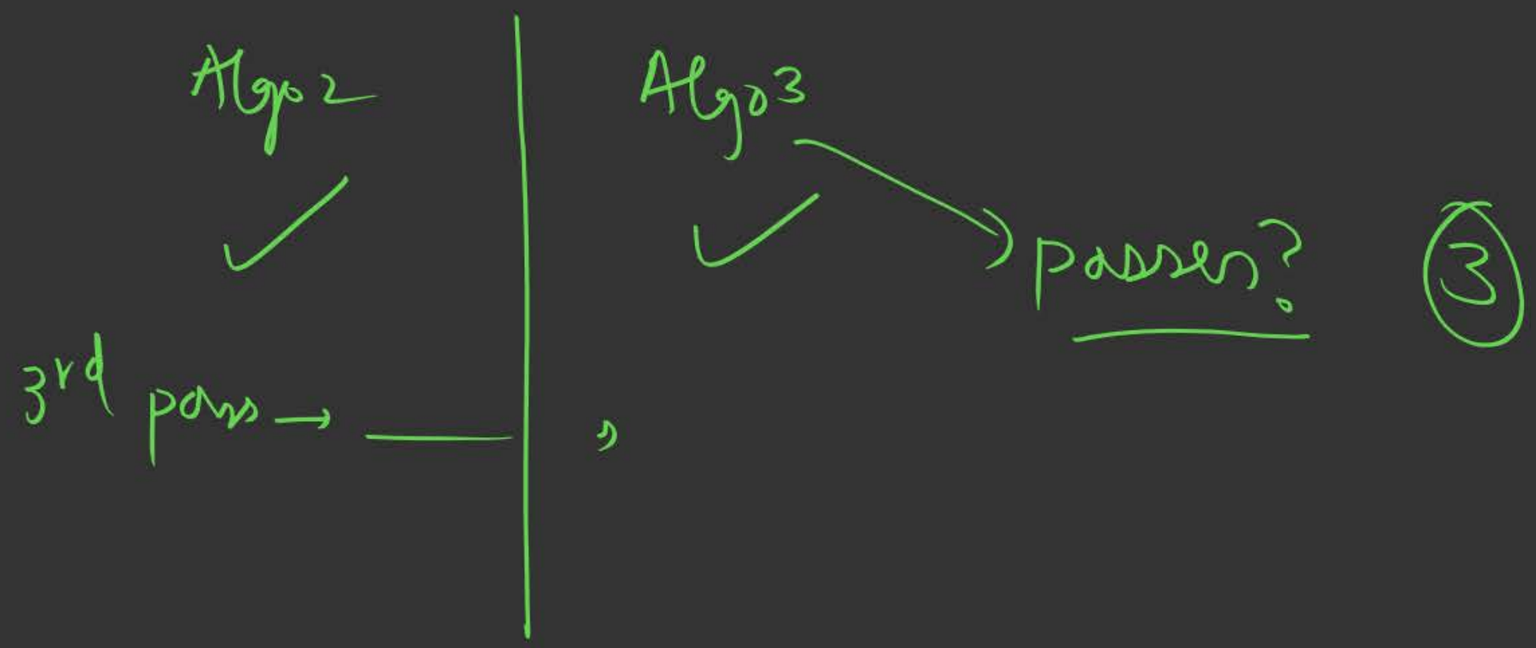
Bonus

---

1) Space Complexity → O(1) → inplace

2) Stable

A = [100 85 10 25 45 50]





Topic : Sorting Algorithms

Code Walkthrough (Algo2 & Algo3):

100	85	10	25	45	50

n = 6 ⟶ 5 passes

Algo2: 5 passes

Pass1: 85 10 25 45 50 100

Pass2: 10 25 45 50 85 100

Pass3: 10 25 45 50 85 100

Pass4: 10 25 45 50 85 100

Pass5: 10 25 45 50 85 100 → 5

100	85	10	25	45	50

Algo3: 3 passes 2X

Pass1: 85 10 25 45 50 100

Pass2: 10 25 45 50 85 100 (Flag=1)

Pass3: 10 25 45 50 85 100 (Flag=0)

→ 0 swaps → exit



Topic : Sorting Algorithms

Selection Sort:

Steps:

1. In i^{th} pass, find the index/position of the ith smallest element.
2. Swap it with the element that is present in the ith index.
3. Keep repeating until array is sorted.



Topic : Sorting Algorithms

Important Observation:

After 1st pass → 1st smallest will be placed at 1st position

After 2nd pass → 2nd smallest will be placed at 2nd position

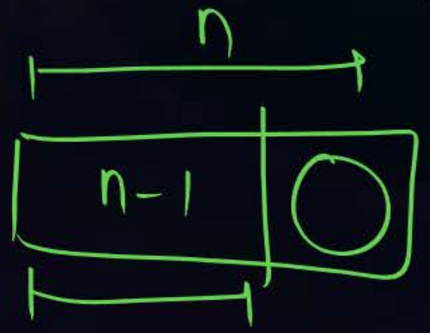
.

.

.

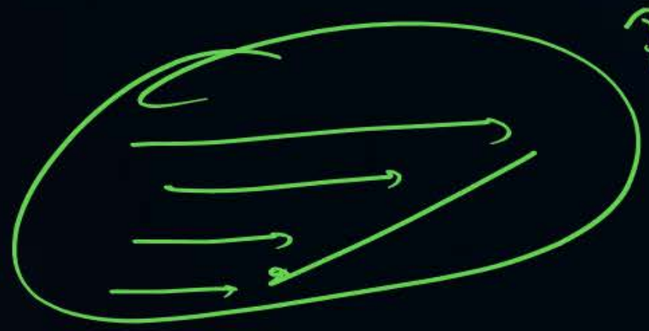
After $(n-1)^{th}$ pass → $(n-1)^{th}$ smallest will be placed at $(n-1)^{th}$ position

→ (n-1) passes





Topic : Sorting Algorithms



Code Walkthrough:

1	2	3	4	5	6
60	90	15	10	30	45

Pass1: 10 | 90 15 60 30 45

Pass2: 10 15 | 90 60 30 45

Pass3: 10 15 30 | 60 90 45

Pass4: 10 15 30 45 | 90 60

Pass5: 10 15 30 45 60 | 90

10 60

A[1] ↔ A[4]

A[2] ↔ A[3]

(n-1) passes always



Topic : Sorting Algorithms

HW

```
Algo SelectionSort(A, n) {
    for (pass = 1; pass <= (n–1); pass++) {
        min-ind = pass                          // initialize
        for (j = pass+1; j <= n; j++) {
            if (A[j] < A[min-ind]) {
                min-ind = j                     // index of min element found so far
            }
        }
        swap(A[pass], A[min-ind])               // place pass^th min at its correct position
    }
}
```

1 3 4

(2 5 10 15]

Comp

P1 : (n-1) comp

P2 : → [n-1] (n-2) comp

P3 → (n-3)

⋮

P(n-1) → 1

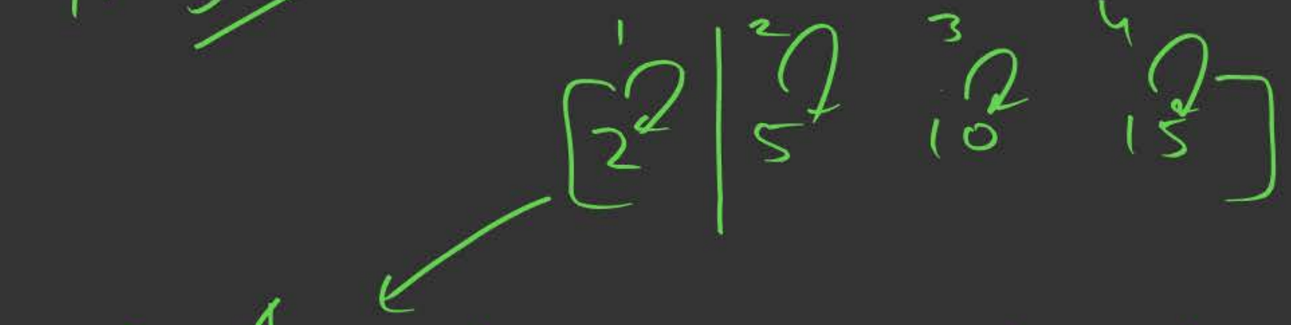
Total Comp :-

1+2+3 . . . (n-1)

$= \boxed{\frac{n(n-1)}{2}}$

irrespective of the i/p ordering.

I/p already sorted



pass 1 : 1 ⟶ 4 min_ind = 1 swap(A[1], A[1])

↳ 2 | 5 10 15

pass 2 : min_ind = 2 2 ⟶ 4 , min_ind = 2 swap(A[2], A[2])

i/p : A[n]

1) Total how many passes = (n-1) passes always

2) How many swaps in 1 pass = 1 swap

3) Total swaps = 1 * (n-1)

= (n-1) swaps



Topic : Sorting Algorithms

Time Complexity analysis of Selection Sort:

$\left[\frac{n(n-1)}{2}\right]$

	No. of Comparisons		No. of Swaps (inversions)	
Best case	$O(n^2)$	+	$(n-1)$	$\rightarrow \Omega(n^2)$
Worst Case	$O(n^2)$	+	$(n-1)$	$\rightarrow \delta(n^2)$

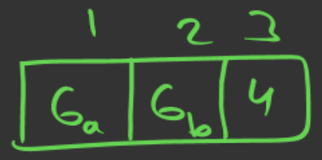
$\Theta(n^2)$



Topic : Sorting Algorithms

Space Complexity analysis of Selection Sort:

$O(1)$ → Constant → {inplace}



pass 1 : $6_a \leftrightarrow 4 \Rightarrow$ 1 2 3 : $4 \mid 6_b \; 6_a$

pass 2 : min_ind = 2 $\Rightarrow 4 \; 6_b \mid 6_a \rightarrow$ O/P

Before Sorting $6_a \mid 6_b \Rightarrow$ After Sorting : $6_b \mid 6_a$

Hence NOT stable.



Topic : Sorting Algorithms

Important Points/Observations:

1. [redacted] Inplace ⟶ SC : O(1)
2. Always taken (n – 1) passes
3. Every pass, exactly 1 swap

   Total = 1 * (n – 1) = (n – 1) swaps to get sorted
4. Among other algorithms, taken least swaps in worst case → (n – 1) → O(n)

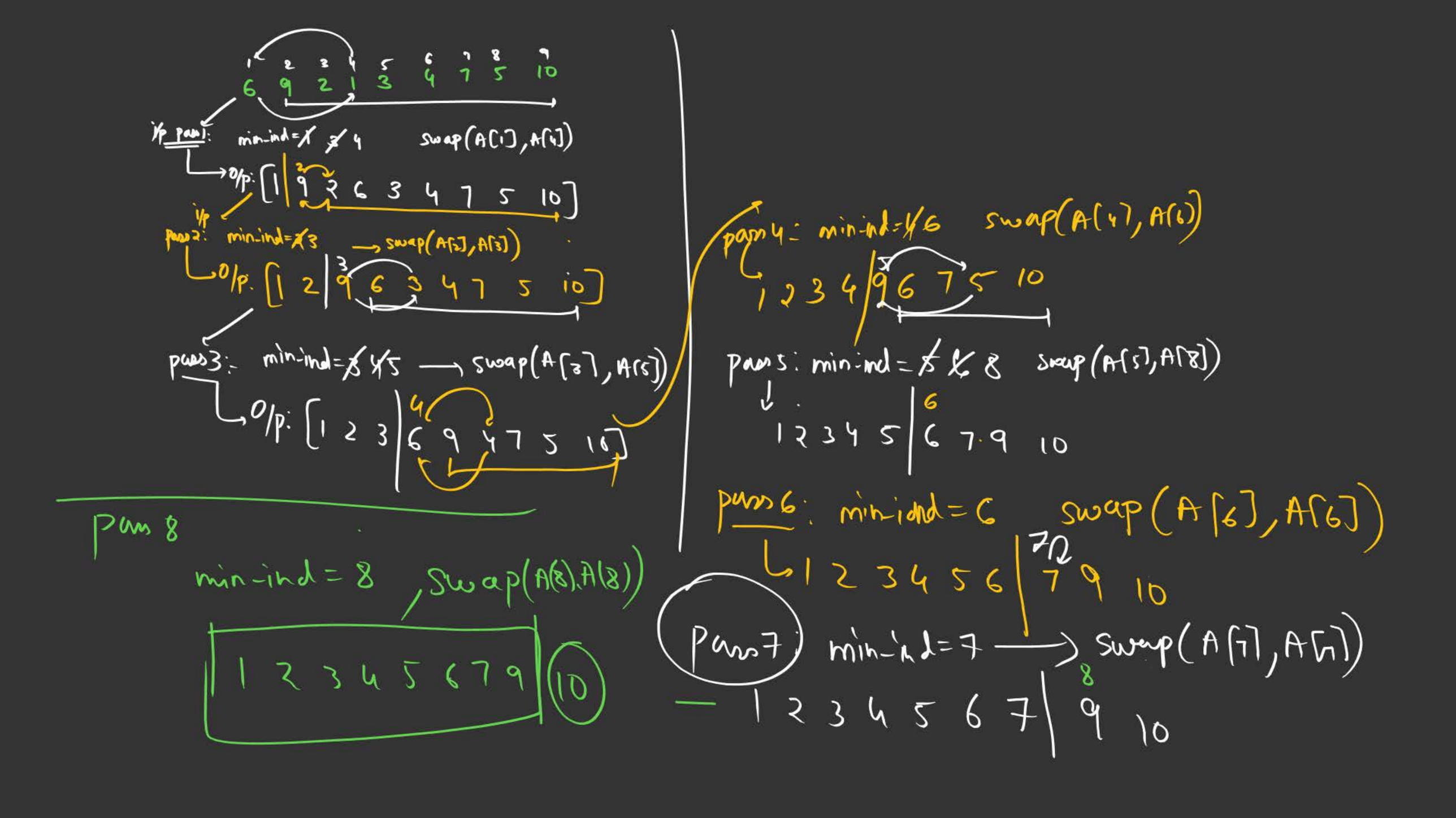
Selection Sort is NOT stable. (unstable)

among all other Comparison based

A = [6 9 2 1 3 4 7 5 10]

perform Selection Sort

& what is O/P after 7th pass ?


1 2 3 4 5 6 7 8 9
6 9 2 1 3 4 7 5 10
i/p pass1: min_ind=1 ⁄ 4 swap(A[1],A[4])
o/p: [1 | 2 9 2 6 3 4 7 5 10]
i/p
pass2: min_ind=3 → swap(A[2],A[3])
o/p: [1 2 | 3 9 6 3 4 7 5 10]
pass3: min_ind=3 ⁄ 5 → swap(A[3],A[5])
o/p: [1 2 3 | 4 6 9 4 7 5 10]
pass4: min_ind=6 swap(A[4],A[6])
1 2 3 4 | 5 9 6 7 5 10
pass5: min_ind=5 ⁄ 8 swap(A[5],A[8])
1 2 3 4 5 | 6 6 7 9 10
pass6: min_ind=6 swap(A[6],A[6])
1 2 3 4 5 6 | 7 7 9 10
pass7 min_ind=7 → swap(A[7],A[7])
1 2 3 4 5 6 7 | 8 9 10
pass 8
min_ind=8 swap(A[8],A[8])
1 2 3 4 5 6 7 9 | 10



2 mins Summary

- Topic
- Topic
- Topic
- Topic
- Topic

Bubble Sort

Selection Sort



THANK - YOU